

# Express Framework

B. Vijay Kumar

# Express Framework : Content

- Introduction to Express
- Installation of Express
- Create first Express application
- request, and response objects
- configuring an Express application
- Rendering views
- sessions
- forms
- file upload
- Connecting to SQL Database

# Introduction

- Express.js is a Node.js web framework that is quick, adaptable, and simple.
- It is essentially a tool that makes it easier to create server-side JavaScript web apps and APIs.

## Why learn Express?

- Express is a user-friendly framework that simplifies the development process of Node applications.
- It uses JavaScript as a programming language and provides an efficient way to build web applications and APIs.
- With Express, you can easily handle routes, requests, and responses, which makes the process of creating robust and scalable applications much easier.

# Introduction

## Features

- **Minimalist Framework:** Express provides a thin layer of fundamental web application features, making it easy to set up a server without a lot of overhead.
- **Middleware Support:** Middleware functions in Express allow you to process requests and responses, handle errors, and add additional functionality. You can use built-in middleware or create your own.
- **Routing:** Express offers a powerful routing system that allows you to define routes for different HTTP methods (GET, POST, PUT, DELETE) and URL patterns.
- **Templating Engines:** You can integrate various templating engines (like Pug, EJS, or Handlebars) to render dynamic HTML views.
- **RESTful API Creation:** Express is ideal for building RESTful APIs, providing the necessary tools to manage routes and handle JSON data.
- **Community and Ecosystem:** It has a large community and a vast ecosystem of middleware and plugins that extend its functionality.

# Installation

## Installing Express involves just a few steps.

### Prerequisites:

- Node.js: Ensure you have Node.js installed.
- npm: Node Package Manager (npm) is installed automatically with Node.js. Check if it's installed

### Steps to Install Express:

- Create a New Directory for Your Project: Open your terminal:

```
mkdir firstapp  
cd firstapp
```

- Initialize a New Node.js Project: Run the following command to create a package.json file:

```
npm init -y
```

# Installation

- Install Express: Use npm to install Express:

```
npm install express
```

- Verify Installation: After installation, you can check your package.json file or the `node_modules` directory to ensure Express is installed.

```
npm list express
```

# First Application

**Once you have done the above task do the following:**

- **Create Your First Express Server:** create a new file app.js and write the following code:

```
const express = require('express');
const app = express();
const PORT = 3000;
app.get('/', (req, res) => {
    res.send('Hello, World!');
});
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

- **Run Your Application:**

```
node app.js
```

# Request and Response Objects

**Request Object (req):** The request object contains information about the incoming HTTP request. It includes various properties and methods that you can use to interact with the request. Check below:

- **req.params:** An object containing route parameters. For example, in the route `/users/:id`, `req.params.id` will contain the value of `id`.
- **req.query:** An object containing the URL query string parameters. For example, in the URL `/search?term=node`, `req.query.term` will contain the value `node`.
- **req.body:** An object containing the parsed request body (for POST requests). To access this, you usually need to use middleware like `express.json()` or `express.urlencoded()`.
- **req.method:** The HTTP method used for the request (GET, POST, etc.).
- **req.url:** The full URL of the request.
- **req.headers:** An object containing the request headers.



# Request and Response Objects

## Example:

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/students/:id', (req, res) => {
  const userId = req.params.id;
  res.send('User ID: ${userId}');
});

app.get('/search', (req, res) => {
  const searchTerm = req.query.term;
  res.send('Search Term: ${searchTerm}');
});

app.listen(PORT, () => {
  console.log('Server is running on http://localhost:${PORT}');
});
```

# Request and Response Objects

Open the URL using postman software to test it as below:

The screenshot displays the Postman application interface. On the left sidebar, the 'My Workspace' section is active, showing a 'Your collection' dialog box with an 'Authorization' dropdown set to 'API Key'. Below this, a message states: 'Create a collection for your requests. A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.' A 'Create Collection' button is visible.

The main workspace shows a GET request to `http://localhost:3000/students/123`. The request is configured with the following tabs: Params, Auth, Headers (6), Body, Scripts, Tests, and Settings. The 'Query Params' section is expanded, showing a table with columns: Key, Value, Description, and Bulk Edit. The table contains two rows, both with 'Key' and 'Value' in the first two columns and 'Description' in the third column.

The response section shows a status of '200 OK' with a response time of '3 ms' and a size of '239 B'. The response body is displayed in 'Pretty' format, showing a single line: `1 User ID: 123`.

# Request and Response Objects

**Response Object (res):** The response object is used to send a response back to the client. It includes various properties and methods that you can use to interact with the request. Check below:

- `res.send(data)`: Sends the HTTP response with the specified data, automatically setting the appropriate content type.
- `res.json(data)`: Sends a JSON response, setting the content type to `application/json`.
- `res.status(code)`: Sets the HTTP status code for the response.
- `res.redirect(url)`: Redirects the client to a different URL.
- `res.render(view, locals)`: Renders a view template (for server-side rendering).

# Request and Response Objects

## Example:

```
const express = require('express');
const app = express();
app.use(express.json()); // Middleware to parse JSON bodies

app.get('/', (req, res) => {
  res.send('Hello, World!');
});

app.get('/user/:id', (req, res) => {
  const userId = req.params.id;
  // Simulate fetching user data
  const user = { id: userId, name: 'Vijay' };
  res.json(user); // Send JSON response
});

app.post('/user', (req, res) => {
  const newUser = req.body; // Get user data from the
  request body
  // Simulate saving user data
  res.status(201).json(newUser); // Send back created user
```

# Request and Response Objects

## Example:(Continued....)

```
});  
app.listen(3000, () => {  
  console.log('Server running on http://localhost:3000');  
});
```

# Configuration

**Configuration:** Configuring an Express application involves setting up the necessary components, middleware, and routes for your application to function properly.

Here are the Steps involved in configuring a basic Express application:

- **Initialize a new Node.js project:** Open your terminal and run the following command to create a new directory and initialize a Node.js project.

```
mkdir myfirstexpressapp  
cd myfirstexpressapp  
npm init -y
```

- Install Express and any other necessary packages using npm.

```
npm install express
```

# Configuration

- **Set Up Routes:** In `routes/index.js`, define a simple route.

```
// routes/index.js
const express = require('express');
const router = express.Router();

// Define a GET route
router.get('/', (req, res) => {
  res.send('Hello, World!');
});

// Export the router
module.exports = router;
```

# Configuration

- **Create Your Express Application:** In app.js, set up the main Express application.

```
// app.js
const express = require('express');
const path = require('path');
const logger = require('morgan'); // Optional logging
    middleware
const indexRouter = require('./routes/index'); // Ensure
    this line is correct

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware
app.use(logger('dev')); // Logs requests to the console
app.use(express.json()); // Parses JSON request bodies
// Parses URL-encoded request bodies
app.use(express.urlencoded({ extended: false }));
```



# Configuration

```
// Serve static files from the 'public' directory
app.use(express.static(path.join(__dirname, 'public')));

// Route setup
app.use('/', indexRouter); // Ensure this is correct

// Error handling middleware (optional)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('Something broke!');
});

// Start the server
app.listen(PORT, () => {
  console.log('Server is running on http://localhost:${
PORT}');
});
```

# Configuration

**Run your application:** In your terminal, execute the following command to start your Express server:

```
node app.js
```

# Rendering Views

**Render Views in Express application:** To render views in express application we need to do the following:

- Install a View Engine(EJS): EJS stands for Embedded JavaScript, is a simple templating language that allows developers to generate HTML markup with plain JavaScript.

```
npm install ejs
```

- Create a directory named views in the root of your project.
- Inside the views directory, create an index.ejs file: and write the following:

```
<!-- views/index.ejs -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title><%= title %></title>
```

# Rendering Views

```
</head>
<body>
<h1><%= message %></h1>
</body>
</html>
```

- **Set Up the View Engine in Express:** Create a new file app.js in root directory with the following code:

```
// app.js
const express = require('express');
const app = express();
const port = 3000;

// Set the view engine to EJS
app.set('view engine', 'ejs');

// Set the directory for views (default is 'views')
app.set('views', './views');
```

# Rendering Views

```
// Routes
app.get('/', (req, res) => {
  res.render('index', { title: 'Home', message: 'Hello,
    World!' });
});

app.listen(port, () => {
  console.log('Server running at http://localhost:${port}');
});
```

- Start the Server and Test:

```
node app.js
```

# Sessions

**Sessions in Express:** Sessions in Express allow you to store user data across requests, which is useful for managing user authentication and other stateful interactions.

- Install express-session:

```
npm install express-session
```

- **Configure Sessions in Express:** Create a new file app.js in root directory and do the following:

```
// app.js
const express = require('express');
const session = require('express-session');
const app = express();
const port = 3000;
```

# Sessions

```
// Configure session middleware
app.use(
  session({
    secret: 'your-secret-key', // Replace with a secure,
    random secret
    resave: false,             // Don't save session if
    unmodified
    saveUninitialized: false,  // Don't create session until
    something is stored
    cookie: {
      maxAge: 60000             // Set cookie expiration time
      in ms (optional)
    }
  })
);

app.get('/', (req, res) => {
  res.send('Session example');
});
```

# Sessions

```
app.listen(port, () => {  
  console.log('Server running at http://localhost:${port}');  
});
```

## Storing and Accessing Session Data:

- Storing Data in the Session:

```
app.get('/login', (req, res) => {  
  // Simulating login by storing user info in the  
  session  
  req.session.user = { username: 'Vijay', role: 'admin'  
    };  
  res.send('User logged in!');  
});
```



# Sessions

- Accessing Session Data

```
app.get('/dashboard', (req, res) => {  
  if (req.session.user) {  
    res.send('Welcome, ${req.session.user.username}!');  
  } else {  
    res.send('Please log in first.');
```

- Destroying Session Data

```
app.get('/logout', (req, res) => {  
  req.session.destroy((err) => {  
    if (err) {  
      return res.send('Error logging out');    }  
    res.send('You are logged out!');  });  
});
```

# Forms

In **Express.js**, forms are a key part of building web applications as they allow users to send data to the server.

## Steps how to implement it:

- Create an sample app handling forms
- npm install express
- create a file app.js
- write the following code:

```
const express = require('express');
const app = express();
const port = 3000;

app.use(express.urlencoded({ extended: true }));
// Parse URL-encoded data
app.use(express.json()); // Parse JSON data
```

# Forms

```
app.get('/', (req, res) => {  
    res.send('<form action="/submit" method="POST"><  
input name="data"><button>Send</button></form>');  
});  
  
app.post('/submit', (req, res) => {  
    res.send('Received: ${req.body.data}');  
});  
  
app.listen(port, () => console.log('App running at  
http://localhost:${port}'));
```

- Now run the code node app.js

# Forms

## Output Screens:

- Default Screen:



A screenshot of a web form's default state. It features a single-line text input field containing the text "vijay". To the right of the input field is a rectangular button with the text "Send". The entire form is set against a light gray background.

- After Submit:

Received: vijay

# Fileupload

Handling file uploads in **Express.js** is straightforward with the help of middleware like multer.

## Steps how to implement it:

- Create an sample app handling forms
- npm install express
- npm install multer : Multer is a middleware for handling multipart/form-data, primarily used for file uploads in express.
- create a file app.js
- write the following code:

```
const express = require('express');  
const multer = require('multer');  
const path = require('path');  
  
const app = express();  
const port = 3000;
```

# Fileupload

```
// Set storage options for Multer
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/'); // Directory where files will
    be saved
  },
  filename: (req, file, cb) => {
    cb(null, Date.now() + path.extname(file.originalname)); // File name
  },
});

// Initialize Multer with storage options
const upload = multer({
  storage: storage,
  limits: { fileSize: 5 * 1024 * 1024 }, // Limit file
  size to 5MB
});
```

# Fileupload

```
// Serve a simple HTML form
app.get('/', (req, res) => {
  res.send(`
    <form action="/upload" method="POST" enctype="
multipart/form-data">
      <input type="file" name="file" />
      <button type="submit">Upload</button>
    </form>
  `);
});

// Route for file upload
app.post('/upload', upload.single('file'), (req, res) => {
  if (!req.file) {
    return res.status(400).send('No file uploaded.');
```

# Fileupload

```
// Start the server
app.listen(port, () => {
  console.log('Server running at http://localhost:${port}');
});
```

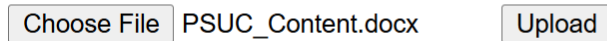
- Now run the code node app.js



# Forms

## Output Screens:

- Default Screen:



Choose File PSUC\_Content.docx Upload

- After Submit:

File uploaded successfully: 1732060900050.docx

## Questions

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ 🔍 ↺

# MongoDB

B. Vijay Kumar

# MongoDB : Content

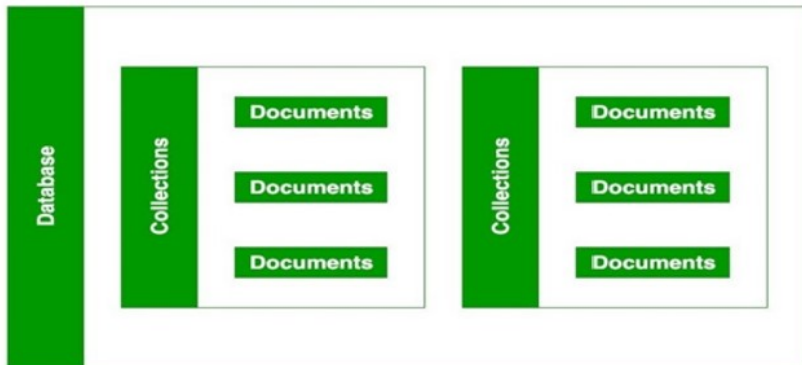
- What is MongoDB?
- Installing MongoDB?
- Collections and Documents
- Using MongoDB Compass
- Using MongoDB Shell
- Creating Database - Mongo Shell
- Adding New documents
- Finding Documents
- Sorting and Limiting Data
- Nested Documents
- Operators
- Updating documents
- Deleting documents

# MongoDB : Content

- Connecting MongoDB using Node.js
- Reading Data from MongoDB
- Writing Data into MongoDB
- Questionnaire

# What is MongoDB?

- MongoDB is an open-source document-oriented database.
- It is categorized under the NoSQL(Not only SQL) database because the storage and retrieval of data in the MongoDB are not in the form of tables.
- The data is stored in the collections and documents.



# Features of MongoDB:

- Schema-less Database
- Document Oriented
- Indexing
- Replication
- Aggregation
- High Performance
- Scalability

## How data stored in MongoDB:

In MongoDB, the basic unit of data is a document. A document is essentially a JSON-like structure composed of field-value pairs. MongoDB uses a binary-encoded format called BSON (Binary JSON) to store these documents.



# Uses of MongoDB:

MongoDB is a popular NoSQL database known for its flexibility, scalability, and performance. It is widely used in various applications across different industries. Such as:

- Content Management System
- E-Commerce Platforms
- Indexing
- Real-Time Analytics
- Internet of things(IOT)
- Gaming Applications
- Customer Relation Management(CRM)
- Social Networks
- Big Data Applications
- Health Care
- Log Management

# Advantages/Disadvantages of MongoDB:

## Advantages:

- It is a schema-less NoSQL database. You need not to design the schema of the database when you are working with MongoDB.
- It provides great flexibility to the fields in the documents.
- It contains heterogeneous data.
- It provides high performance, scalability.
- It is a document oriented database and the data is stored in BSON documents.

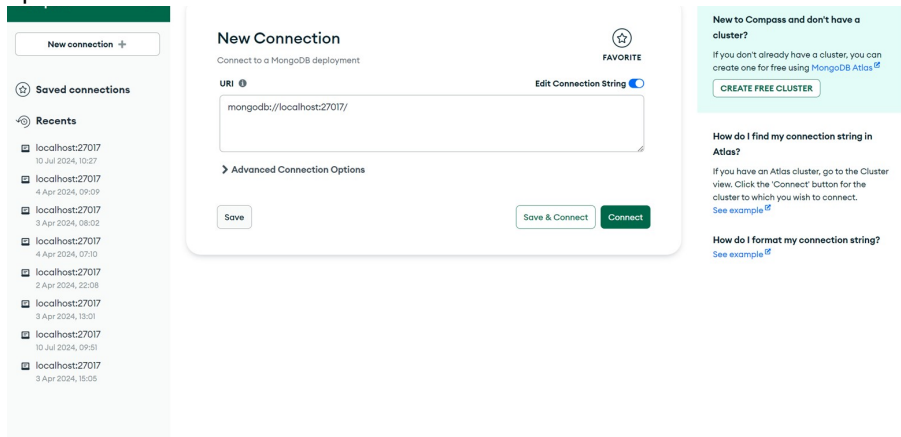
## Disadvantages:

- It uses high memory for data storage.
- We are not allowed to store more than 16MB data in the documents.
- The nesting of data in BSON is also limited you are not allowed to nest data more than 100 levels.

# Installing MongoDB

## MongoDB Community Server:(Gives us MongoDB and Compass(GUI)) Download Link:

<https://www.mongodb.com/try/download/community-kubernetes-operator>



**New connection** +

**Saved connections**

- localhost:27017  
10 Jul 2024, 10:27
- localhost:27017  
4 Apr 2024, 09:09
- localhost:27017  
3 Apr 2024, 08:02
- localhost:27017  
4 Apr 2024, 07:10
- localhost:27017  
2 Apr 2024, 22:08
- localhost:27017  
3 Apr 2024, 13:01
- localhost:27017  
10 Jul 2024, 09:51
- localhost:27017  
3 Apr 2024, 15:05

**New Connection**

Connect to a MongoDB deployment

URI ⓘ Edit Connection String 

mongodb://localhost:27017/

**Advanced Connection Options**

Save Save & Connect Connect

**FAVORITE**

**New to Compass and don't have a cluster?**

If you don't already have a cluster, you can create one for free using [MongoDB Atlas](#)

[CREATE FREE CLUSTER](#)

**How do I find my connection string in Atlas?**

If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.

[See example](#)

**How do I format my connection string?**

[See example](#)

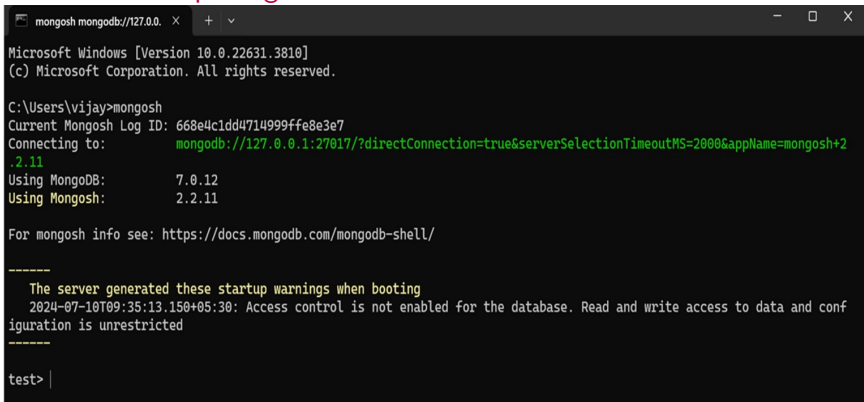
# Installing MongoDB

## MongoDB Shell:(CLI)

### Download Link:

<https://www.mongodb.com/try/download/shell>

**Note: Download package msi**



```
mongosh mongodb://127.0.0.1 x + v
Microsoft Windows [Version 10.0.22631.3810]
(c) Microsoft Corporation. All rights reserved.

C:\Users\vijay>mongosh
Current Mongosh Log ID: 668e4c1dd4714999ffe8e3e7
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.2.11
Using MongoDB:      7.0.12
Using Mongosh:       2.2.11

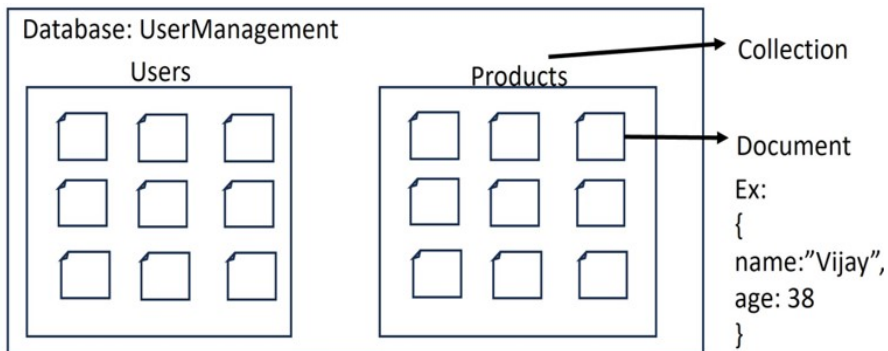
For mongosh info see: https://docs.mongodb.com/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2024-07-10T09:35:13.150+05:30: Access control is not enabled for the database. Read and write access to data and conf
  igation is unrestricted
-----

test> |
```

# Collections and Documents

## MongoDB Database Representation:

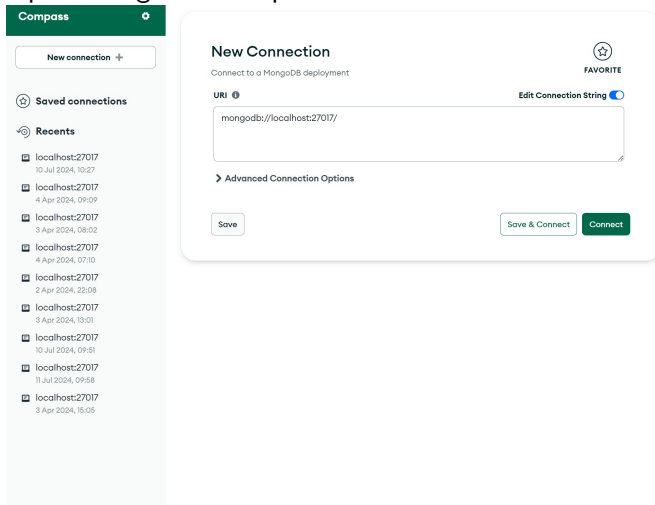


# Collections and Documents

- **Database:** In MongoDB, A database is an organized collection of structured or unstructured information stored electronically on a machine locally or in the cloud.
- **Collections:** A collection is a grouping of MongoDB documents. Documents within a collection can have different fields.
- **Documents:** A document in MongoDB is a basic unit of data that is stored in JSON-like format (BSON – Binary JSON). Documents contain key-value pairs and can have nested documents and arrays.

# Using MongoDB Compass

## • Open MongoDB Compass From Startmenu



### New to Compass and don't have a cluster?

If you don't already have a cluster, you can create one for free using [MongoDB Atlas](#)

[CREATE FREE CLUSTER](#)

### How do I find my connection string in Atlas?

If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.

[See example](#)

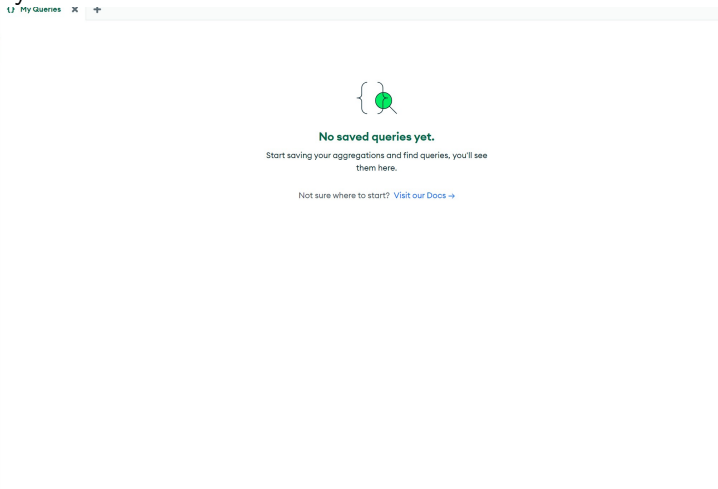
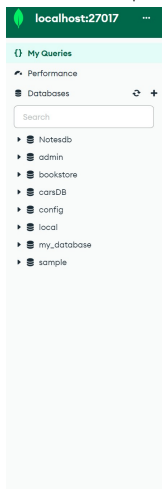
### How do I format my connection string?

[See example](#)

## • Click on Connect to Interact with MongoDB Server

# Using MongoDB Compass

- Click on '+' Symbol on Left menu to create a database





# Using MongoDB Compass

## Create Database ✕

### Database Name

### Collection Name

☐ **Time-Series**

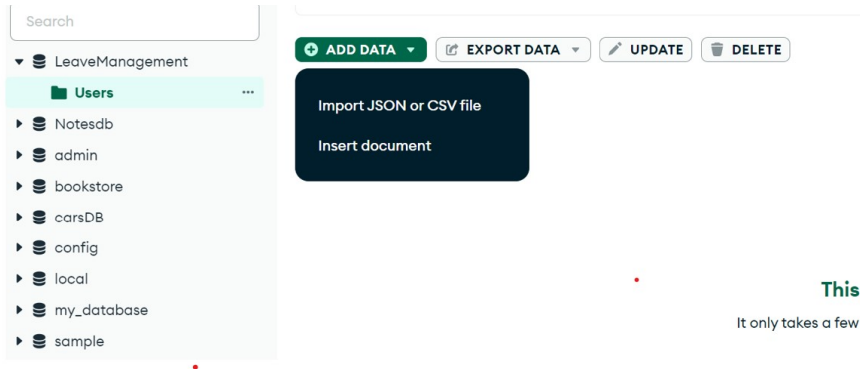
Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

➤ **Additional preferences** (e.g. Custom collation, Capped, Clustered collections)

CancelCreate Database

# Using MongoDB Compass

- Select the collection of the database from the left menu
- Click on Add Data to add a new document/documents



We can add a new document by selecting Insert Document or  
We can import the JSON or CSV file to insert the documents

# Using MongoDB Compass

## Default screen when we click on Insert Document

To collection LeaveManagement.Users

VIEW  

1 /\*\*  
2 \* Paste one or more documents here  
3 \*/  
4 {  
5    "\_id": {  
6      "\$oid": "669e5773e462014a350ebbf5"  
7    }  
8 }

Cancel Insert

We can add a Single document or we can add multiple documents

# Using MongoDB Compass

## Example: Adding Single Document



An Id will inserted automatically when we insert a document into the collection See below:



```

_id: ObjectId('669e5a17e462014a350ebbf8')
name: "Vijay"
city: "Hyd"
    
```

# Using MongoDB Compass

We can insert multiple documents, multiple documents can be added as an array by selecting Insert Document from Add Data Menu check below:

## Insert Document

To collection LeaveManagement.Users

VIEW



```
1  /**
2  * Paste one or more documents here
3  */
4  [
5  {
6    "name": "Ajay",
7    "city": "Hyd"
8  },
9  {
10   "name": "Shiva",
11   "city": "Mumbai"
12 },
13 ]
14
```

Cancel

Insert

# Using MongoDB Compass

## User collection after inserting 2 documents

```
_id: ObjectId('669e5a17e462014a350ebbf8')  
name: "Vijay"  
city: "Hyd"
```

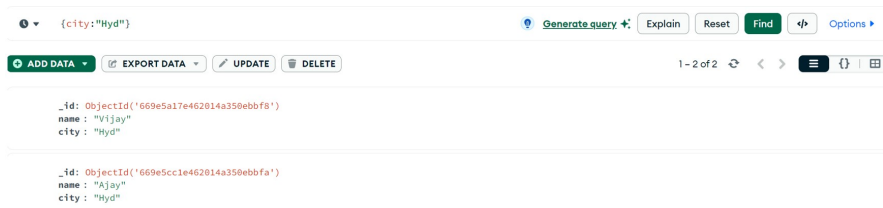
```
_id: ObjectId('669e5cc1e462014a350ebbf9')  
name: "Ajay"  
city: "Hyd"
```

```
_id: ObjectId('669e5cc1e462014a350ebbf9')  
name: "Shiva"  
city: "Mumbai"
```

**We can update or delete the document by selecting the specific document**

# Using MongoDB Compass

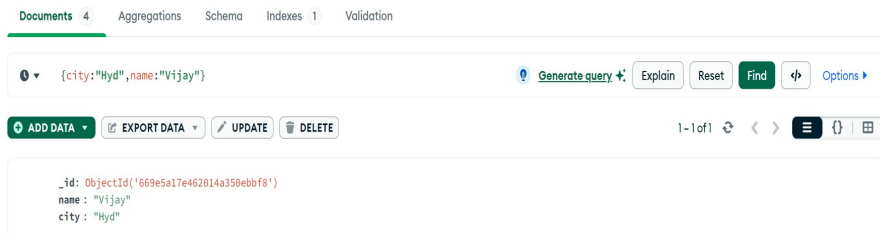
We can filter the documents by giving a query in the query box given above the collection as shown below:



In the above collection we are filtering the documents with city "Hyd"

# Using MongoDB Compass

We can also give multiple queries to filter the documents



The screenshot shows the MongoDB Compass interface. At the top, there are tabs for Documents (4), Aggregations, Schema, Indexes (1), and Validation. The Documents tab is selected. Below the tabs, there is a query filter input field containing the JSON query: `{city:"Hyd",name:"Vijay"}`. To the right of the input field are buttons for [Generate query](#), [Explain](#), [Reset](#), [Find](#), and [Options](#). Below the query field, there are buttons for [ADD DATA](#), [EXPORT DATA](#), [UPDATE](#), and [DELETE](#). On the right side, there is a status bar showing "1 - 1 of 1" and navigation icons. Below the status bar, a document is displayed with the following fields: `_id: ObjectId('669e5a17e462014a350ebbf8')`, `name: "Vijay"`, and `city: "Hyd"`.

In the above collection we are filtering the documents with city "Hyd" and name "Vijay"



# Using MongoDB Shell

## We can use Mongo Shell in two ways

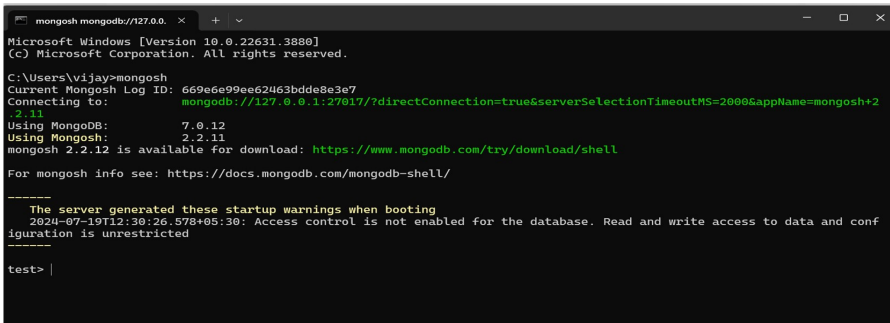
- We can directly use Mongosh from MongoDB Compass by selecting at the bottom as shown below:.

The screenshot displays the MongoDB Compass web interface. On the left, a sidebar lists databases and collections, with 'Users' selected under the 'LeaveManagement' database. The main area shows a query filter: `{city: "Hyd", name: "Vijay"}`. Below the query, there are buttons for 'ADD DATA', 'EXPORT DATA', 'UPDATE', and 'DELETE'. The results section shows one document: `{_id: ObjectId('669e5a17e462014a350ebbfb'), name: "Vijay", city: "Hyd"}`. At the bottom, the 'MONGOOSH' terminal is open, showing the prompt `est> |`.

# Using MongoDB Shell

We can also use command prompt to use mongo shell

- Open command prompt
- type mongosh



```
Microsoft Windows [Version 10.0.22631.3880]
(c) Microsoft Corporation. All rights reserved.

C:\Users\vijay>mongosh
Current Mongosh Log ID: 669e6e99ee62463bdde8e3e7
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.2.11
Using MongoDB:      7.0.12
Using Mongosh:       2.2.11
mongosh 2.2.12 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://docs.mongodb.com/mongodbs-shell/

-----
The server generated these startup warnings when booting
2024-07-19T12:30:26.578+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> |
```

# Creating Database - Mongo Shell

- Open command prompt
- Navigate to mongo shell prompt by typing the command `mongosh`
- Use the following command either to switch or create a new database **use databasename**

Ex: use usermanagement

```
Current Mongosh Log ID: 669f1f03c481e7db7fe8e3e7
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2
.2.11
Using MongoDB:      7.0.12
Using Mongosh:       2.2.11
mongosh 2.2.12 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://docs.mongodb.com/mongosh-shell/

=====
  The server generated these startup warnings when booting
  2024-07-23T04:31:07.025+05:30: Access control is not enabled for the database. Read and write access to data and conf
  igation is unrestricted
=====

test> use usermanagement
switched to db usermanagement
usermanagement>
```

**Note:** Database and Collection will be created permanently by inserting at least one document.

# Adding New Documents

**Adding the documents to Collection can be done in two ways:**

1. `insertOne()`: Insert one document into the collection

```
1 db.users.insertOne(  
2 {  
3   name:"Vijay",  
4   city:"Hyd",  
5   department:"CSE"  
6 }  
7 )  
8
```

```
usermanagement> db.users.insertOne({name:"Vijay",city:"Hyd",department:"CSE"})  
{  
  acknowledged: true,  
  insertedId: ObjectId('669f22ffc481e7db7fe8e3e9')  
}  
usermanagement> |
```

An ObjectId will be generated automatically after inserting a new document

# Adding New Documents

## 2. insertMany(): Insert Multiple documents into the collection

```
1 db.users.insertMany(  
2 [  
3 {name:"Ajay",city:"Hyd",department:"CSE"},{name:"Shiva",  
city:"Wgl",department:"ECE"}  
4 ]  
5 )  
6
```

```
usermanagement> db.users.insertMany([{name:"Ajay",city:"Hyd",department:"CSE"},{name:"Shiva",city:"Wgl",department:"ECE"}])  
{  
  acknowledged: true,  
  insertedIds: {  
    '0': ObjectId('669f2638c481e7db7fe8e3ea'),  
    '1': ObjectId('669f2638c481e7db7fe8e3eb')  
  }  
}  
usermanagement> |
```

# Finding Documents

## We use find() method for finding the documents

```
1 db.users.find()  
2
```

This method gives us first 20 documents of the collection.  
Note: If we want more no.of documents we need to type **it** at the mongosh to get more 20 documents like this it goes on...

```
1 db.users.find({department:"CSE"})  
2
```

Filters the data based on the single search criteria

```
1 db.users.find({department:"CSE",city:"Hyd"})  
2
```

Filters the data based on multiple search criteria

# Finding Documents

```
1 4.      db.users.find({department:"CSE",city:"Hyd"},{name:1,  
2         department:1})
```

Retrieve the specific fields of the documents based on search criteria

```
1 5.      db.users.find({}, {name:1, department:1})  
2
```

Retrieve the specific fields of all the documents with empty search criteria

**findOne():** we can also use findOne() method to get only one document from the search results

```
1      db.users.findOne({department:"CSE"})  
2
```

**show collections:** Display all the collections in the database

# Sorting and Limiting Data

**sort():** It is the method used to sort the documents ascending or descending order

*Ascending order:*

```
1 db.users.find().sort({name:1})  
2
```

*Descending order:*

```
1 db.users.find().sort({name:-1})  
2
```

**count():** It is the method used to get the no. of documents from the collection

```
1 db.users.count()  
2
```



# Sorting and Limiting Data

**limit():** It is the method used to get the limited no.of documents from the collection.

```
1 db.users.find().limit(3)
```

**We can also use chaining of methods as below:**

```
1 db.users.find({department:"CSE"}).limit(2)
```

```
1 db.users.find({department:"CSE"}).count()
```

# Nested Documents

**In MongoDB we can embed/nest a document inside another document**

Syntax:

```
1  {  
2  ....  
3  field: {field1: value1, field2: value2}  
4  ....  
5  }  
6
```

Example

```
1  db.users.insertOne(  
2  {  
3  name:{firstname:"Vijay",lastname:"kumar"},  
4  city:"Hyd",  
5  department:"CSE"  
6  }  
7  )  
8
```

# Operators

- MongoDB provides us the operators for filtering the documents with complex queries
- Every operator in MongoDB to be prefixed with \$ sign
- **\$or:** It is an operator used to filter the document with multiple search criteria, it is similar to or condition in programming. languages

```
1 Ex: db.users.find({$or:[{name:"Vijay"},{city:"Hyd"}]})
```

```
2
```

- **\$in:** It is an operator used to filter the documents in between the multiple values for a single field.

```
1 Ex: db.users.find({city:{$in: ["Hyd","Wgl"]}})
```

```
2
```

- **\$nin:** It is an operator used to filter the documents not in between the multiple values for a single field.

```
1 Ex: db.users.find({city:{$nin: ["Hyd"]}})
```

```
2
```

# Operators

- **\$gt:** It is an operator used to filter the documents with single field or multiple fields greater than the specified value.

```
1 Ex: db.users.find({age:{$gt: 32}})
```

```
2
```

- **\$gte:** It is an operator used to filter the documents with single field or multiple fields greater than or equal to the specified value.

```
1 Ex: db.users.find({age:{$gte: 32}})
```

```
2
```

- **\$lt:** It is an operator used to filter the documents with single field or multiple fields lesser than the specified value.

```
1 Ex: db.users.find({age:{$lt: 32}})
```

```
2
```

- **\$lte:** It is an operator used to filter the documents with single field or multiple fields lesser than or equal to the specified value.

```
1 Ex: db.users.find({age:{$lte: 32}})
```

```
2
```

# Updating Documents

**Updating the documents to Collection can be done in two ways:**

1. `updateOne()`: It is the method used to update one document in the collection

- **\$set**: It is an operator used to update the specific field or multiple field values based on the filter criteria

```
1
2 db.users.updateOne({_id:ObjectId('668
   e55bbd4714999ffe8e3e9')},{ $set:{name:"Vijay Kumar",
   age:35}})
3
```

# Updating Documents

- **\$inc:** It is an operator used to increment the field value by the given number.

```
1 db.users.updateOne({_id:ObjectId('668  
   e55bbd4714999ffe8e3e9')}},{ $inc:{age:5}})
```

- **\$push:** It is an operator used to push an element into the array of the field in the document of a collection.

```
1 db.users.updateOne({_id:ObjectId('668  
   e55bbd4714999ffe8e3e9')}},{ $push:{reviews:{title:"  
   review3",body:"excellent"}}})
```

- **\$pull:** It is an operator used to pull an element from an array field in the document of a collection.

```
1 db.users.updateOne({_id: ObjectId('669  
   f42d7c481e7db7fe8e3ee')}},{ $pull:{hobbies:"h3"}})
```

# Updating Documents

1. `updateMany()`: It is the method used to update the multiple documents based on the search criteria.

- **\$set:**

```
1 db.users.updateMany({name:"shiva"},{$set: {name:"Shiva"  
    }})  
2  
3 db.users.updateMany({name:"Shiva"},{$set: {name:"  
    s h i v a ,age:25}})  
4
```

- **\$inc:**

```
1 db.users.updateMany({name:"shiva"},{$inc: {age:2}})  
2
```

- **\$push:**

```
1 db.users.updateMany({name:"shiva"},{$push: {hobbies:  
    reading    }})  
2
```

# Updating Documents

- **\$pull:**

```
1 db.users.updateMany({name:"shiva"},{$pull: {hobbies:  
      reading      }})
```

2



# Deleting Documents

**Deleting the documents to Collection can be done in two ways:**

1. `deleteOne()`: This method is used to delete one document from a collection.

```
1 db.users.deleteOne({_id: ObjectId('668  
   f62557442aa754eeb7f83')})
```

1. `deleteMany()`: This method is used to delete one or more documents from a collection.

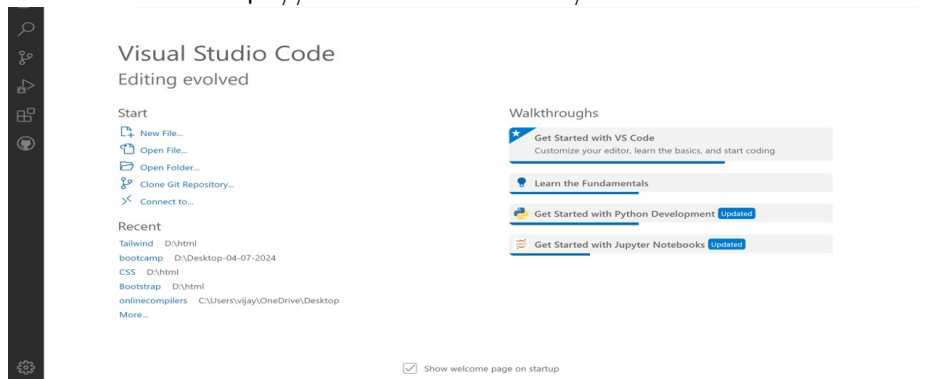
```
1 db.users.deleteMany({age:32})  
2  
3 db.users.deleteMany({age:32,name:    vijay    })
```

# Connecting to MongoDB using Nodejs

## Step 1:

Install Visual code editor using following link

Download Link: <https://code.visualstudio.com/>



# Connecting MongoDB using Nodejs

## Step 2:

We need to install Node.js and npm(Node package manager)

Download Link: <https://nodejs.org/en/download/prebuilt-installer>

## Download Node.js®

Download Node.js the way you want.

Package Manager **Prebuilt Installer** Prebuilt Binaries Source Code

I want the **v20.15.1 (LTS)** version of Node.js for **Windows** running **x64**

 **Download Node.js v20.15.1**

Node.js includes npm (10.7.0) ↗.

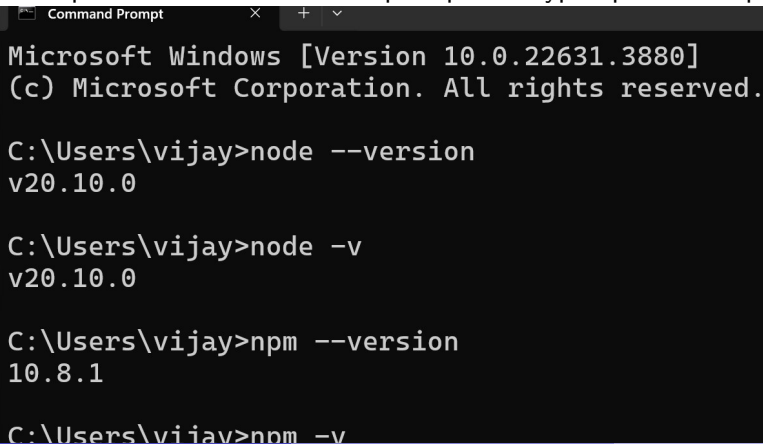
[Read the changelog for this version](#) ↗

# Connecting to MongoDB using Nodejs

## Step 3:

### Verify Installation

- Node: Go to the command prompt and type `node -v` or `node --version`
- npm: Go to the command prompt and type `npm -v` or `npm --version`



```
Microsoft Windows [Version 10.0.22631.3880]
(c) Microsoft Corporation. All rights reserved.

C:\Users\vijay>node --version
v20.10.0

C:\Users\vijay>node -v
v20.10.0

C:\Users\vijay>npm --version
10.8.1

C:\Users\vijay>npm -v
```

# Connecting to MongoDB using Nodejs

**Mongoose:** Inorder to connect to MongoDB using Nodejs we need to install mongoose

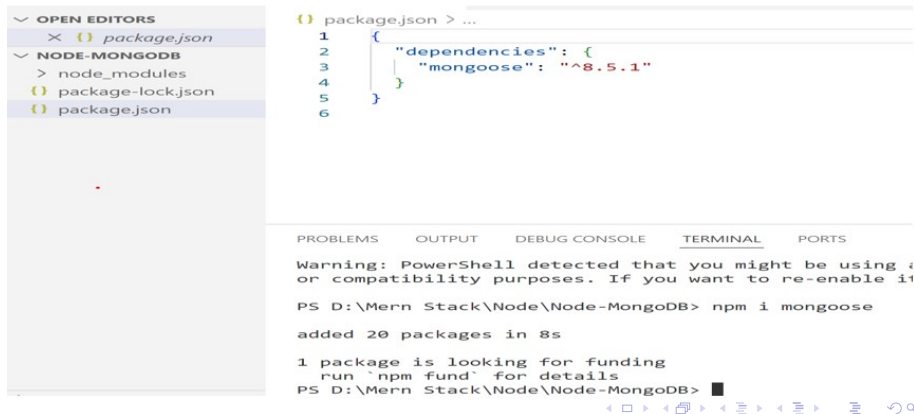
- Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js.
- It is used to give schema-based solution to model your application data.
- Mongoose, you can define schemas for your documents, which are then translated into MongoDB collections.

# Connecting to MongoDB using Nodejs

## Install Mongoose:

Open visual studio code terminal and type the following command

```
npm i mongoose
```



```
{ package.json > ...
1  {
2    "dependencies": {
3      "mongoose": "^8.5.1"
4    }
5  }
6
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Warning: PowerShell detected that you might be using it
or compatibility purposes. If you want to re-enable it
PS D:\Mern Stack\Node\Node-MongoDB> npm i mongoose
added 20 packages in 8s
1 package is looking for funding
  run `npm fund` for details
PS D:\Mern Stack\Node\Node-MongoDB>
```

# Connecting to MongoDB using Nodejs

## Steps to connect to MongoDB:

- Create a project folder in the machine.
- Open project folder using Visual studio Code.
- Create a new file called app.js
- Copy the database URL from mongoDB compass installed in the machine.
- Write the following code to connect to MongoDB

```
1
2 const mongoose=require("mongoose");
3 mongoose.connect("mongodb://localhost:27017/
   usermanagement").then( ()=>{
4     console.log("connected");
5   });
6
```

- Run the project using the following command  
`node app`

# Connecting to MongoDB using Nodejs

Check the below screen shot for the reference:

The screenshot shows the VS Code interface with the Explorer, Open Editors, and Node-MongoDB panels on the left. The main editor displays the code in `app.js`. The terminal at the bottom shows the command `node app` being executed, resulting in the output `connected`.

```
EXPLORER
  JS app.js
  X

OPEN EDITORS
  X JS app.js

NODE-MONGODB
  > node_modules
  JS app.js
  {} package-lock.json
  {} package.json

JS app.js > ...
1  const mongoose=require("mongoose");
2  mongoose.connect("mongodb://localhost:27017/usermanagement").then(()=>{
3    console.log("connected");
4  });

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Mern Stack\Node\Node-MongoDB> node app
connected
```

:

- Database is usermanagement
- If we connect to MongoDB successfully we get "connected" message at the console.



# Connecting to MongoDB using Nodejs

Check the below screen shot for the reference:

The screenshot shows the VS Code interface with the Explorer, Open Editors, and Node-MongoDB panels. The Explorer panel shows the file structure with 'app.js' selected. The Open Editors panel shows 'app.js' open. The Node-MongoDB panel shows the connection status as 'connected'. The terminal panel shows the command 'node app' executed, resulting in the output 'connected'.

```
EXPLORER
...
JS app.js X

OPEN EDITORS
X JS app.js

NODE-MONGODB
> node_modules
JS app.js
{} package-lock.json
{} package.json

JS app.js > ...
1  const mongoose=require("mongoose");
2  mongoose.connect("mongodb://localhost:27017/usermanagement").then(()=>{
3    console.log("connected");
4  });

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\Mern Stack\Node\Node-MongoDB> node app
connected
```

:

- Database is usermanagement
- If we connect to MongoDB successfully we get "connected" message at the console.

# Reading Data from MongoDB

In order to read or write the data into the collection we need to create Schema and Model of the collection in the database

## Example: users collection

```
1  const Schema = mongoose.Schema;
2  const userSchema = new Schema({
3      name: String,
4      city: String,
5      department: String
6  });
7  const User = mongoose.model('User', userSchema);
8
```

# Reading Data from MongoDB

Here is the sample code to read the data from the users collection of usermanagement database  
Create a new file readdata.js and write the following code in it

```
1 const mongoose = require('mongoose');
2 const mongoURI = 'mongodb://localhost:27017/usermanagement';
   // Replace with your connection string
3 mongoose.connect(mongoURI)
4 .then(() => {
5     console.log('MongoDB connected successfully.');
```

```
6     const Schema = mongoose.Schema;
7     const userSchema = new Schema({
8         name: String,
9         city: String,
10        department: String
11    });
```

# Reading Data from MongoDB

Sample Code continued...

```
1 const User = mongoose.model('users', userSchema);
2   User.find()
3     .then(users => {
4       console.log('Users:', users);
5     })
6     .catch(err => {
7       console.error('Error fetching users:', err);
8     });
9
10 })
11 .catch((err) => {
12   console.error('MongoDB connection error:', err);
13 });
14
```

Execute the project using the following command  
node readdata

# Read Data from MongoDB

## output:

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS D:\Mern Stack\Node\Node-MongoDB> node readdata
MongoDB connected successfully.
Here's is the users data
Users: [
  {
    _id: new ObjectId('669f22ffc481e7db7fe8e3e9'),
    name: 'Vijay',
    city: 'Hyd',
    department: 'CSE'
  },
  {
    _id: new ObjectId('669f2638c481e7db7fe8e3ea'),
    name: 'Ajay',
    city: 'Hyd',
  }
]
```

# Writing Data to MongoDB

In order to read or write the data into the collection we need to create Schema and Model of the collection in the database

## Example: users collection

```
1  const Schema = mongoose.Schema;
2  const userSchema = new Schema({
3      name: String,
4      city: String,
5      department: String
6  });
7  const User = mongoose.model('User', userSchema);
8
```

# Writing Data to MongoDB

Here is the sample code to write the data into the users collection of usermanagement database

Create a new file writedata.js and write the following code in it

```
1 const mongoose = require('mongoose');
2 const mongoURI = 'mongodb://localhost:27017/usermanagement';
   // Replace with your connection string
3 mongoose.connect(mongoURI)
4 .then(() => {
5     console.log('MongoDB connected successfully.');
```

// Define a schema

```
7 const Schema = mongoose.Schema;
8 const userSchema = new Schema({
9     name: String,
10    city: String,
11    department:String
12 });
```

# Writing Data to MongoDB

Sample Code continued...

```
1 // Create a model
2 const userModel = mongoose.model('users', userSchema);
3 // Use the model to create a document
4 const newUser = new userModel({
5   name: 'Ramu',
6   city: "Hyd",
7   department: "ECE"
8 });
9 // Save the document to the database
10 newUser.save()
11 .then(doc => {
12   console.log('Document saved:', doc);
13 })
14 .catch(err => {
15   console.error('Error saving document:', err);
16 });
17 })
18 .catch((err) => {
```



# Writing Data to MongoDB

Sample Code continued...

```
1 .catch((err) => {  
2     console.error('MongoDB connection error:', err);  
3 });  
4
```

Execute the project using the following command  
node writedata

**output:**

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS D:\Mern Stack\Node\Node-MongoDB> node writedata  
MongoDB connected successfully.  
Document saved successfully: {  
  name: 'Ramu',  
  city: 'Hyd',  
  department: 'ECE',  
  _id: new ObjectId('66a1e858c0933a638a942f86'),  
  __v: 0  
}
```

# Writing Data to MongoDB

If we observe in the output screenshot there is a new field

```
1  __v:0  
2
```

is added automatically by mongoose while adding a new document into the collection.

- This key is used by Mongoose to track the version of each document.
- We can exclude this by giving versionKey: false while creating the mongoose schema as shown below:

```
1 const Schema = mongoose.Schema;  
2 const userSchema = new Schema({  
3   name: String,  
4   city: String,  
5   department:String  
6 },{versionKey:false});  
7
```

# Sample Code

## Github Repository

<https://github.com/bolluvijaykumar/nodejs-mongodb.git>

### Steps: To run the sample code

- 1 Download the zip file from the repository
- 2 Extract and open the firstproject folder using VS Code
- 3 Open terminal and execute the following command to install the node and mongoose  
`npm i mongoose`
- 4 Run the command "node app" to test connecting to mongodb database
- 5 Run the command "node writedata" to test writing data to the users collection
- 6 Run the command "node readdata" to test reading data from the users collection

# Questionnaire

## Small Answer Questions

- 1 How does MongoDB store data?
- 2 How are collections and documents related in MongoDB?
- 3 What features does MongoDB Compass provide for managing databases?
- 4 What command is used to show all databases in MongoDB Shell?
- 5 How do you switch to a specific database in MongoDB Shell?

## Long Answer Questions

- 1 Explain the key features and benefits of using MongoDB as a NoSQL database compared to traditional relational databases.
- 2 Describe the relationship between databases, collections, and documents in MongoDB. How does this hierarchy differ from the table-based structure of relational databases?
- 3 Describe the key features of MongoDB Compass. How can Compass be used to perform operations like creating collections, inserting documents, querying data, and visualizing schema?
- 4 Describe the steps involved in setting up a connection to MongoDB from a Node.js application. What packages are commonly used, and how are they configured?
- 5 Describe the process of reading data from a MongoDB database using Mongoose in a Node.js application. What methods are available, and how are they used?

A set of navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.